

# Loyal Spark

Loyalty-as-a-Service on Base - for merchants, customers, and AI agents

*Sentient Foundation Open Source AGI Grant Application - Supporting Document*

## Executive summary

---

Loyal Spark is Loyalty-as-a-Service (LaaS) on Base Mainnet. Merchants launch full onchain loyalty programs (ERC-20 token, minting, rewards, vouchers, CRM) through a web portal - without building Web3 infrastructure from scratch. Customers hold points in their own wallets. Developers and autonomous AI agents use the same backend via REST API and MCP (Model Context Protocol).

The codebase is MIT-licensed on GitHub. Smart contracts are live on Base. We expose open agent discovery (agent.json, OpenAPI, llms.txt, 13 Markdown skills) and support wallet-only agent registration (SIWE) with no dashboard required. Paid agent corridors use x402 micropayments (USDC on Base).

## At a glance

---

- Live on Base Mainnet (Chain ID 8453)
- Merchant portal + customer portal (Privy, SIWE, embedded wallets)
- Agent API: 23 REST routes + 32 merchant MCP tools + 14 recipient MCP tools
- x402 and MPP pay-per-call gateways
- Capacitor native apps scaffolded (iOS / Android - grant target: store release)
- Open source: [github.com/aspekt19/unboxed-loyalty-spark](https://github.com/aspekt19/unboxed-loyalty-spark) (MIT)

## What the \$25,000 grant unlocks (4-6 months)

---

Bootstrapping, I can keep mainnet running - but I cannot properly ship mobile apps, gasless UX, and open ecosystem work at the same time. This grant funds focused delivery:

### 1. App Store + Google Play

Ship two Capacitor apps already scaffolded in the repo: Loyal Spark Business (merchants) and Loyal Spark (customers). Includes store assets, QA, review cycles, and dev account costs. Goal: loyalty that normal businesses can adopt as a mobile app, not only a website.

### 2. Paymaster on Base (gasless transactions)

Integrate a Paymaster so merchants and customers can mint, redeem, and transfer without holding ETH for gas. Wire through existing Privy + embedded wallet flows. Document the integration pattern in the MIT repo for other builders.

### 3. Open agent layer

- Expand the public skills bundle (merchant + holder end-to-end flows)
- Reference MCP client configs (Cursor, Claude Desktop)
- Polish SIWE agent registration (lsk\_ API keys without UI)
- x402 paid MCP/API examples; target 10+ unique external paying agent wallets

### 4. Merchant pilots + live infra

- Onboard 2-3 pilot merchants: deploy, mint, voucher - web + mobile + gasless
- 6 months production infra (RPC, Supabase, monitoring) so API/MCP stay up for integrators

*Without the grant: mainnet stays up, but mobile, Paymaster, open docs, and merchant pilots stall - hurting anyone building on our open repo.*

## Problem

---

- Legacy loyalty is closed: customers do not truly own points
- Small businesses cannot afford custom Web3 builds or \$99-499/mo legacy SaaS with revenue cuts
- Going onchain often ends as a bare token with no vouchers, tiers, or operations
- AI agents need the same economic layer - issue value, reward users, settle payments - but lack a ready stack

## Solution - Loyalty-as-a-Service

---

One platform on Base where any merchant can launch a full loyalty program in minutes - same infrastructure for everyone, not a custom build per brand.

|                           |                                                                    |
|---------------------------|--------------------------------------------------------------------|
| <b>Merchants (portal)</b> | Deploy token, mint, rewards, vouchers, team, CRM                   |
| <b>Customers (wallet)</b> | Own tokens, redeem, transfer, P2P escrow marketplace               |
| <b>Agents (API + MCP)</b> | Automate programs; pay per call via x402 or API keys (lsk_ / rwk_) |

## Why now

---

- Cheap L2 transactions and USDC on Base
- Wallet abstraction (Privy) lowers the bar for non-crypto merchants and customers
- MCP and x402 make agent-to-agent and pay-per-request commerce practical
- Missing piece: an open, composable loyalty layer - not another one-off token per project

## Product - live today

---

### Merchant flow

- Deploy ERC-20 loyalty token via factory contract
- Mint to email, phone, or wallet; vouchers, tiers, RFM, automation
- Invite team (branches, employees) and AI agents (scoped API keys)

### Customer flow

- Receive tokens to embedded or external wallet
- Redeem rewards; trade on P2P escrow marketplace
- Holder-side API (rwk\_) and recipient MCP for agent-driven redemption

### Agent / developer surface

- REST agent-api + loyalty-mcp (merchant) + recipient-loyalty-mcp (holder)
- Autonomous registration: SIWE message, no web login required
- CDP MPC server wallets for autonomous onchain operations
- x402-gateway and mpp-gateway for pay-per-request access

## What's open (MIT on GitHub)

---

- Full application and Edge Function source
- 13 step-by-step agent skills (skills/loyal-spark/)
- MCP examples, OpenAPI, agent.json, llms.txt
- Contract ABIs, addresses, autonomous agent registration docs

If we shut down tomorrow: merchants lose a working LaaS stack; customers lose live programs; developers lose running API/MCP and x402 endpoints. The repo can be forked, but production continuity would break.

## Smart contracts - Base Mainnet

---

|                                 |                                            |
|---------------------------------|--------------------------------------------|
| <b>LoyaltyTokenFactory</b>      | 0x5F3DdBa12580CFdc6016258774cCc19C4250dA80 |
| <b>LoyalSparkERC20 (impl)</b>   | 0xe6BA426C9c51281B929a17444De02c65815E27C3 |
| <b>LoyaltyTokenEscrow (P2P)</b> | 0xA569C95AfC1BCF381c48BcF336ED9D2c014bcdDF |
| <b>Builder Code (ERC-8021)</b>  | bc_wdmnog7m                                |

## Team

---

Solo founder - built Loyal Spark as LaaS on Base, then added the agent layer (REST, MCP, x402, CDP wallets) on the same backend. Shipped contracts and product on mainnet. Ships in public on GitHub (MIT).

## Grant request

---

- Track: Grant (non-dilutive)
- Amount: \$25,000 USD
- Focus: mobile store release, Paymaster, open agent docs, merchant pilots

## Demo & trial links

---

|                                 |                                                                                                                   |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Website:</b>                 | <a href="https://loyalspark.online">https://loyalspark.online</a>                                                 |
| <b>Merchant portal:</b>         | <a href="https://loyalspark.online/merchant">https://loyalspark.online/merchant</a>                               |
| <b>For agents / onboarding:</b> | <a href="https://loyalspark.online/for-agents">https://loyalspark.online/for-agents</a>                           |
| <b>API documentation:</b>       | <a href="https://loyalspark.online/api-docs">https://loyalspark.online/api-docs</a>                               |
| <b>Agent manifest:</b>          | <a href="https://loyalspark.online/.well-known/agent.json">https://loyalspark.online/.well-known/agent.json</a>   |
| <b>GitHub (MIT):</b>            | <a href="https://github.com/aspekt19/unboxed-loyalty-spark">https://github.com/aspekt19/unboxed-loyalty-spark</a> |

## Contact

---

Website: <https://loyalspark.online>

Email: [admin@loyalspark.online](mailto:admin@loyalspark.online)

X: [https://x.com/Loyal\\_Spark](https://x.com/Loyal_Spark)

*(c) 2026 Loyal Spark. MIT-licensed open source. Built on Base.*